SIMATS ENGINEERING

DESIGN AND ANALYSIS OF ALGORITHMS

Assignment 4 – Part I

Algorithmic Design through Pseudocode – Worked Solutions

Task 1 – Algorithm Design Review

Problem type: packages are divisible and the objective is to maximise loaded value under a weight limit. This is the **fractional knapsack** problem, which is solved optimally by a greedy rule on value-per-unit-weight.

Task 1.1 – Diagnose the Design

#	Deficiency (where)	Why another engineer cannot implement it reliably
1	**Inputs and outputs undefined** – "Read packages", "RETURN result"	It does not say that each package has a weight and a value, nor that a vehicle capacity exists. The output could be a total value, a list of packages or fractions; the implementer must guess.
2	**No ordering key** – "Arrange packages"	Arrange by weight, value, ID or ratio? Ascending or descending? Different choices give different totals, so two engineers produce different programs.
3	**"Best" is undefined** – "Select the best package"	Best may mean highest value, lightest weight or highest value/weight. Only highest value/weight is optimal for a divisible load; the pseudocode never commits to it, so the greedy-choice rule is hidden.
4	**Selection done once, outside the loop**	The best package is chosen a single time before the WHILE loop; nothing says how the next package is picked, or that a loaded package is removed, so the same package may be added again.
5	**Capacity / remaining capacity not modelled** – "vehicle has space"	There is no variable holding the remaining capacity, no initial value and no numeric test (Remaining > 0). The loop condition cannot be evaluated.
6	**Feasibility condition missing** – "suitable package"	"Suitable" is not tested against anything. The comparison Weight ≤ Remaining is never stated.
7	**Partial package not handled**	The problem allows division, but the algorithm never says when to split, what fraction to take (Remaining / Weight) or how much value that fraction earns.

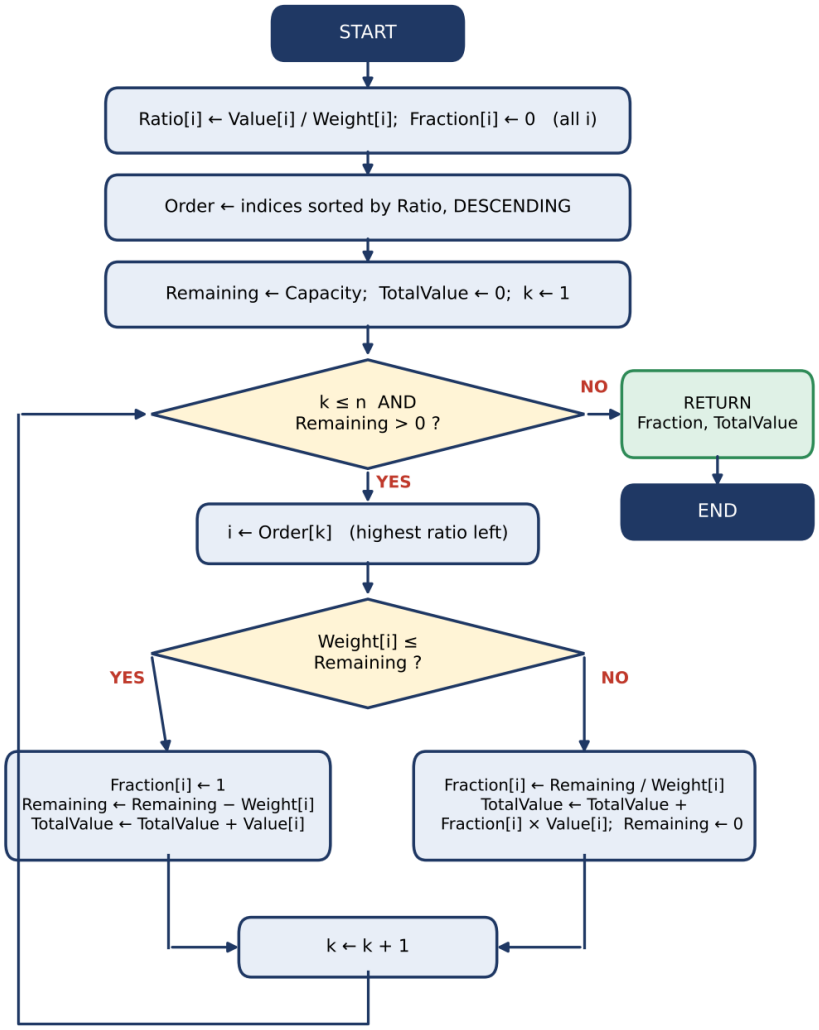
#	Deficiency (where)	Why another engineer cannot implement it reliably
8	**Remaining capacity never updated**	Nothing subtracts the loaded weight, so the loop condition never changes and the loop can run forever.
9	**Termination condition incomplete**	If all packages are used before the vehicle is full, the loop still waits for space to run out. There is no "packages exhausted" test or index.
10	**Result construction vague** – "Calculate value"	No accumulator, no formula, and no record of which package was loaded in what fraction, so the loading plan cannot be reported.

Task 1.2 – Repair the Design

Greedy-choice rule: always load, as much as possible of, the unloaded package with the **highest value-per-unit-weight (Value / Weight)**.

```
ALGORITHM FractionalLoad(n, Weight[1..n], Value[1..n], Capacity)
INPUT:
  n        - number of packages (n ≥ 1)
  Weight[i] - weight of package i (Weight[i] > 0)
  Value[i]  - value of package i (Value[i] ≥ 0)
  Capacity  - vehicle capacity (Capacity ≥ 0)
OUTPUT:
  Fraction[1..n] - fraction of each package loaded (0 ≤ Fraction[i] ≤ 1)
  TotalValue     - maximum total value loaded
BEGIN
  FOR i ← 1 TO n DO
    Ratio[i] ← Value[i] / Weight[i]
    Fraction[i] ← 0
  END FOR
  Order ← indices 1..n SORTED in descending order of Ratio
  Remaining ← Capacity
  TotalValue ← 0
  k ← 1
  WHILE k ≤ n AND Remaining > 0 DO
    i ← Order[k]                    // greedy choice: highest ratio left
    IF Weight[i] ≤ Remaining THEN   // package fits completely
      Fraction[i] ← 1
      Remaining ← Remaining − Weight[i]
      TotalValue ← TotalValue + Value[i]
    ELSE                            // only part fits: fill the vehicle
      Fraction[i] ← Remaining / Weight[i]
      TotalValue ← TotalValue + Fraction[i] × Value[i]
      Remaining ← 0
    END IF
    k ← k + 1
  END WHILE
  RETURN Fraction, TotalValue
END
```

Fig. 1 Flowchart of the repaired greedy algorithm (Task 1.2)



Quick verification. Capacity = 50; packages (weight, value): (10, 60), (20, 100), (30, 120). Ratios are 6, 5 and 4. Load package 1 fully (remaining 40), package 2 fully (remaining 20), then 20/30 of package 3, giving 60 + 100 + 80 = **240**, which is the known optimum.

Complexity: O(n log n) for sorting plus O(n) for the loop. **Termination:** k increases every iteration, so the loop ends after at most n iterations; it also stops as soon as Remaining reaches 0.

Task 1.3 – Design Defence (≤ 100 words)

The rewritten solution is pseudocode because it declares inputs and outputs, uses explicit variables (Remaining, Fraction, TotalValue) and states every decision as a precise condition: sort by Value/Weight in descending order, compare Weight[i] with Remaining, and take Remaining/Weight[i] when a package does not fit. It uses only language-neutral constructs (FOR, WHILE, IF, ←) and the abstract SORT, with no syntax, type declarations, libraries or input/output calls of C, Java or Python. Unlike informal English, every step is unambiguous, ordered and terminating, so any programmer can implement it directly.

Task 2 – Job Sequencing with Deadlines

Task 2.1 – Model the Requirement

Element	Description
Input	n jobs; for each job: Job ID, Deadline d (a positive integer slot number), Profit p. Each job needs exactly one unit slot.
Output	A schedule (which job occupies which slot) and the total profit earned.
Objective	Maximise the sum of profits of the jobs that are completed on or before their deadline.
Constraints	(1) One job per slot. (2) A job may run only in a slot t with $1 \leq t \leq$ its deadline. (3) Each job is used at most once. (4) A late or unscheduled job earns nothing.
Decision	For every job: accept it and choose a slot, or reject it.
Information needed	Which slots 1..MaxDeadline are already occupied (a Slot[] array), and the job's deadline, so we can search for a free slot at or before the deadline.

Task 2.2 – Greedy Hypothesis

Greedy rule: consider jobs in descending order of profit and place each job in the **latest free slot not later than its deadline**, rejecting it if no such slot exists.

Why it is plausible: (a) processing by profit means a more valuable job is never blocked by a less valuable one that was accepted earlier; (b) choosing the *latest* free slot keeps the early slots free for later jobs with tight deadlines, so an accepted job uses as little "flexibility" as possible. An exchange argument shows that any optimal schedule can be transformed into the one produced by this rule without losing profit.

Task 2.3 – Algorithm Design

```
ALGORITHM JobSequencing(n, JobID[1..n], Deadline[1..n], Profit[1..n])
INPUT:  n jobs with JobID, Deadline (≥ 1) and Profit (≥ 0)
OUTPUT: Slot[1..MaxD] (job placed in each slot), TotalProfit
BEGIN
  SORT the jobs in descending order of Profit          // ordering criterion
  MaxD ← maximum of Deadline[1..n]
  FOR t ← 1 TO MaxD DO
    Slot[t] ← EMPTY                                    // slot availability
  END FOR
  TotalProfit ← 0
  FOR k ← 1 TO n DO                                    // k-th most profitable job
    t ← Deadline[k]                                    // start at the deadline
    WHILE t ≥ 1 AND Slot[t] ≠ EMPTY DO                 // feasibility check
      t ← t − 1                                        // move to an earlier slot
    END WHILE
    IF t ≥ 1 THEN                                      // a free slot exists
      Slot[t] ← JobID[k]                               // job placement
      TotalProfit ← TotalProfit + Profit[k]            // profit update
    ELSE
      REJECT JobID[k]                                  // no feasible slot
    END IF
  END FOR
  RETURN Slot[1..MaxD], TotalProfit                    // final schedule and profit
END
```

Complexity: O(n log n) for sorting plus O(n × MaxD) for slot searching in the worst case.

Task 2.4 – Desk-Check

Sorted by profit: A(2, 80), C(2, 60), D(3, 40), B(1, 35), E(1, 25). MaxD = 3. Slots: [– , – , –].

Step	Job (d, p)	Slots examined	Decision	Slot[1..3] after	TotalProfit
1	A (2, 80)	Slot 2 – free	Place A in slot 2	– , A , –	80
2	C (2, 60)	Slot 2 – taken; slot 1 – free	Place C in slot 1	C , A , –	140
3	D (3, 40)	Slot 3 – free	Place D in slot 3	C , A , D	180
4	B (1, 35)	Slot 1 – taken; t becomes 0	REJECT (no slot)	C , A , D	180
5	E (1, 25)	Slot 1 – taken; t becomes 0	REJECT (no slot)	C , A , D	180

Result: schedule C (slot 1) → A (slot 2) → D (slot 3), **total profit = 180**. Check: the best alternative using slot 1 for B gives B + A + D = 155, which is lower.

Task 2.5 – Challenge Your Design

Change made: profit of B is changed from 35 to **90** (all other values unchanged).

1. Prediction: Yes, the schedule will change. B will now be the most profitable job, so it is processed first and takes slot 1.

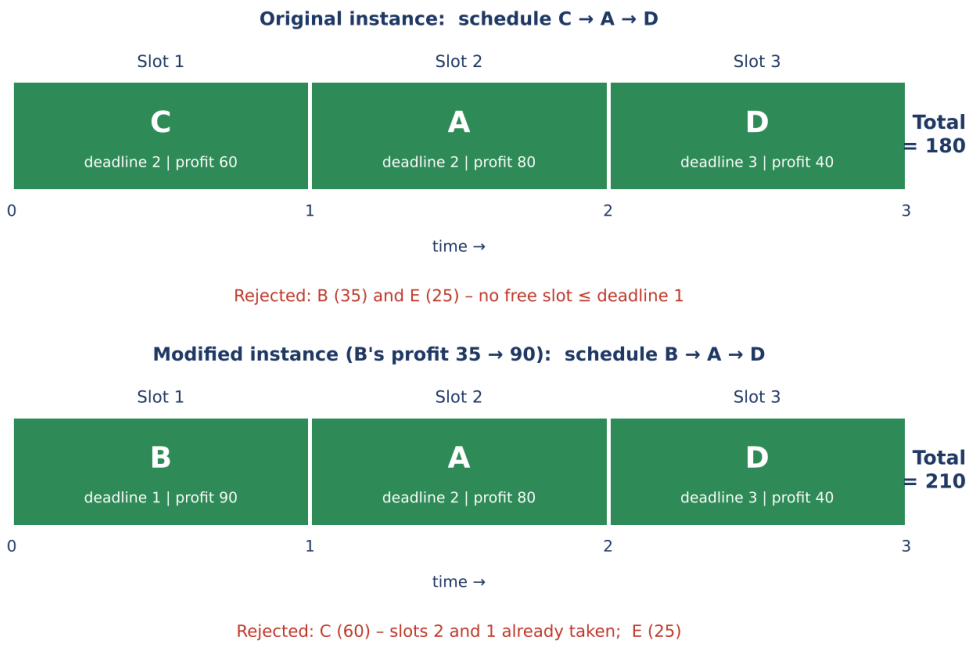
2. Decision expected to be affected: the placement of C. Previously C took slot 1; now slot 1 is occupied by B (and slot 2 by A), so C should have no free slot and be rejected.

3. Trace of relevant decisions: new order is B(1, 90), A(2, 80), C(2, 60), D(3, 40), E(1, 25).

Step	Job	Decision	Slots after	TotalProfit
1	B (1, 90)	Slot 1 free → place B	B , – , –	90
2	A (2, 80)	Slot 2 free → place A	B , A , –	170
3	C (2, 60)	Slot 2 taken, slot 1 taken → REJECT	B , A , –	170
4	D (3, 40)	Slot 3 free → place D	B , A , D	210

The prediction was **correct**: the schedule becomes B → A → D with profit 210 (= 180 − 60 + 90). The affected decision was C's feasibility check.

Fig. 2 Final schedules produced by the greedy algorithm (Task 2.4 and 2.5)



Task 3 – Dynamic Programming: 0/1 Knapsack

Task 3.1 – Define the State

DP[i][w] = the **maximum total value** that can be obtained by choosing items only from the **first i items** (items 1..i), each used at most once, such that the **total weight does not exceed w**. Here $0 \le i \le n$ and $0 \le w \le W$, and the answer to the whole problem is DP[n][W].

Task 3.2 – Develop the Recurrence

Case	Condition	Recurrence	Meaning
Base 1	i = 0	DP[0][w] = 0	With no items available, no value can be obtained.
Base 2	w = 0	DP[i][0] = 0	With zero capacity nothing fits (all weights are positive).
Cannot include	Weight[i] > w	DP[i][w] = DP[i−1][w]	Item i is too heavy for capacity w, so the best is whatever the first i−1 items achieve.
Exclude	Weight[i] ≤ w	DP[i−1][w]	Skip item i: value is the best of the first i−1 items with the same capacity.
Include	Weight[i] ≤ w	Value[i] + DP[i−1][w − Weight[i]]	Take item i: gain Value[i], and the remaining capacity w − Weight[i] is filled optimally using the first i−1 items.
Decision	Weight[i] ≤ w	DP[i][w] = max(Exclude , Include)	The better of the two choices is kept.

Combined: DP[i][w] = DP[i−1][w] if Weight[i] > w; otherwise DP[i][w] = max(DP[i−1][w] , Value[i] + DP[i−1][w − Weight[i]]).

Task 3.3 – Pseudocode

```
ALGORITHM Knapsack01(n, ItemID[1..n], Weight[1..n], Value[1..n], W)
INPUT:  n items; Weight[i] positive integer; Value[i] ≥ 0; capacity W (integer)
OUTPUT: MaxValue, Selected (list of chosen item IDs)
BEGIN
  // ---- initialisation (boundary conditions) ----
  FOR w ← 0 TO W DO  DP[0][w] ← 0  END FOR
  FOR i ← 1 TO n DO  DP[i][0] ← 0  END FOR
  // ---- fill table: row i needs only row i−1 ----
  FOR i ← 1 TO n DO
    FOR w ← 1 TO W DO
      Exclude ← DP[i−1][w]
      IF Weight[i] ≤ w THEN
        Include ← Value[i] + DP[i−1][w − Weight[i]]
        IF Include > Exclude THEN
          DP[i][w] ← Include
        ELSE
          DP[i][w] ← Exclude
        END IF
      ELSE
        DP[i][w] ← Exclude                  // item i cannot be included
      END IF
    END FOR
  END FOR
  MaxValue ← DP[n][W]                       // optimum value
  // ---- reconstruct the chosen items ----
  Selected ← EMPTY LIST
  w ← W
  FOR i ← n DOWN TO 1 DO
    IF DP[i][w] ≠ DP[i−1][w] THEN           // value changed, so item i was taken
      ADD ItemID[i] to Selected
      w ← w − Weight[i]
    END IF
  END FOR
  RETURN MaxValue, Selected
END
```

Complexity: time O(n × W), space O(n × W) (the full table is kept so that the items can be reconstructed).

Task 3.4 – Desk-Check

Items: P(2, 12), Q(3, 20), R(4, 25), S(5, 30); W = 7. Completed DP table (shaded cells lie on the reconstruction path):

	w=0	w=1	w=2	w=3	w=4	w=5	w=6	w=7
i=0 (none)	0	0	0	0	0	0	0	0
i=1 (P: 2, 12)	0	0	12	12	12	12	12	12
i=2 (Q: 3, 20)	0	0	12	20	20	32	32	32
i=3 (R: 4, 25)	0	0	12	20	25	32	37	45

	w=0	w=1	w=2	w=3	w=4	w=5	w=6	w=7
i=4 (S: 5, 30)	0	0	12	20	25	32	37	45

Key state computations

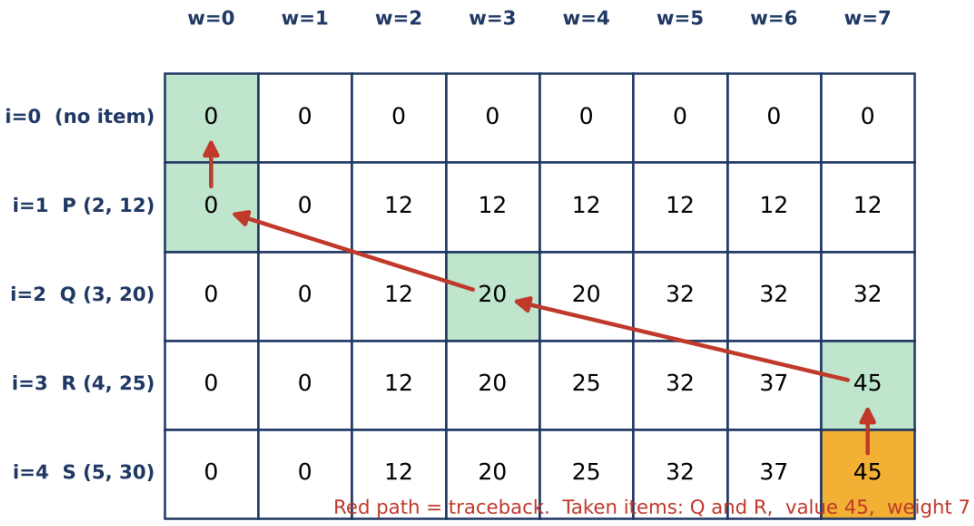
State	Exclude	Include	Chosen
DP[2][3]	DP[1][3] = 12	20 + DP[1][0] = 20	20 (Q taken)
DP[2][5]	DP[1][5] = 12	20 + DP[1][2] = 32	32 (P + Q)
DP[3][4]	DP[2][4] = 20	25 + DP[2][0] = 25	25 (R taken)
DP[3][6]	DP[2][6] = 32	25 + DP[2][2] = 37	37 (P + R)
DP[3][7]	DP[2][7] = 32	25 + DP[2][3] = 45	45 (Q + R)
DP[4][7]	DP[3][7] = 45	30 + DP[3][2] = 42	45 (S not taken)

Reconstruction (w = 7):

i	Compare	Result	w after
4	DP[4][7] = 45 vs DP[3][7] = 45	Equal → S not selected	7
3	DP[3][7] = 45 vs DP[2][7] = 32	Different → **R selected**	7 − 4 = 3
2	DP[2][3] = 20 vs DP[1][3] = 12	Different → **Q selected**	3 − 3 = 0
1	DP[1][0] = 0 vs DP[0][0] = 0	Equal → P not selected	0

Optimum value = 45; selected items = Q and R (total weight 7).

Fig. 3 DP table DP[i][w] with traceback (Task 3.4)



Task 3.5 – Explain the Dependency

Every state DP[i][w] is defined through two earlier states: DP[i−1][w] (exclude) and DP[i−1][w − Weight[i]] (include). Both belong to row **i−1** and to a capacity **not larger** than w. Their values must

already be final before DP[i][w] can be evaluated; otherwise the max() would compare incomplete numbers.

This is why the pseudocode (1) initialises row 0 and column 0 first, (2) uses the **outer loop i ← 1 TO n in increasing order**, so a whole row is complete before the next row is started, and (3) can run the inner loop on w in any order, because no state reads another cell of its own row. The reconstruction loop works in the opposite direction (i from n down to 1) because it follows the dependencies backwards from DP[n][W].

Task 4 – Pseudocode Forensics: Word Break

Convention: substring(Text, j, i) is the piece of Text from position j+1 to i (length i − j). DP[i] should mean "the first i characters of Text can be completely segmented into dictionary words".

Task 4.1 – Locate the Defect

Defective decision: the test IF substring(Text, j, i) is in Dictionary inside the inner loop.

Why it is incomplete: a split at j divides the prefix of length i into two parts: the prefix Text[1..j] and the last piece Text[j+1..i]. The dictionary check only proves that the **last piece** is a word. It says nothing about whether the **first j characters** can themselves be segmented, which is exactly what DP[j] records. The prefix of length i is segmentable only if **DP[j] = TRUE AND the last piece is in the Dictionary**. Without the DP[j] check, a valid-looking last word can sit behind an unsegmentable prefix and wrongly set DP[i] to TRUE.

Task 4.2 – A Failing Input

Text = "xyzcat" (n = 6), Dictionary = { cat, dog }. Correct answer: FALSE, because the prefix "xyz" cannot be segmented.

i	Pieces tested (j → substring)	In Dictionary?	DP[i] (faulty)
0	–	–	TRUE (base)
1	0→"x"	No	FALSE
2	0→"xy", 1→"y"	No, No	FALSE
3	0→"xyz", 1→"yz", 2→"z"	No, No, No	FALSE
4	"xyzc", "yzc", "zc", "c"	None	FALSE
5	"xyzca", "yzca", "zca", "ca", "a"	None	FALSE
6	0→"xyzcat", 1→"yzcat", 2→"zcat", **3→"cat"**	**Yes at j = 3**	**TRUE (WRONG)**

The **first incorrect state is DP[6]**. The algorithm sets it TRUE because "cat" is in the dictionary, although DP[3] = FALSE (the prefix "xyz" is not segmentable). The faulty algorithm therefore returns TRUE instead of FALSE.

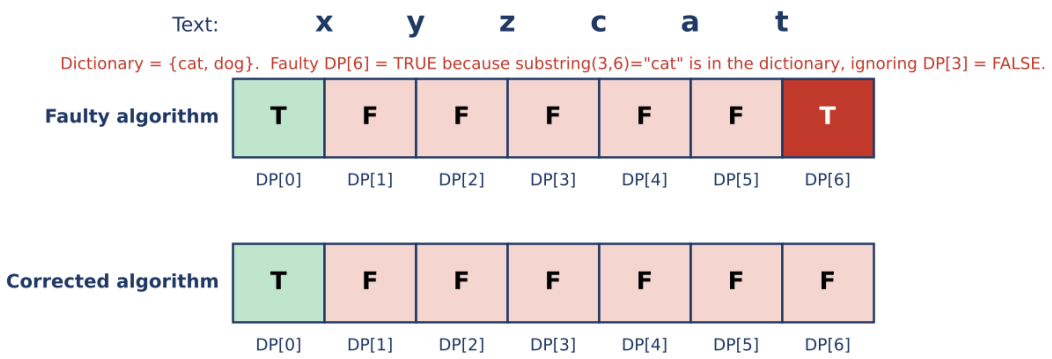
Task 4.3 – Repair the Algorithm

Only the condition inside the inner loop is changed; everything else (initialisation, loop bounds, BREAK, RETURN) is unchanged.

```
    FOR j ← 0 TO i − 1 DO
-     IF substring(Text, j, i) is in Dictionary THEN          // original (faulty)
+     IF DP[j] = TRUE AND substring(Text, j, i) is in Dictionary THEN   // corrected
        DP[i] ← TRUE
        BREAK
      END IF
    END FOR
```

Re-trace of the failing input: at i = 6, j = 3 the test now evaluates DP[3] = FALSE, so the condition fails. The remaining splits j = 4 ("at") and j = 5 ("t") are not in the dictionary, so DP[6] stays FALSE and the algorithm correctly returns FALSE.

Fig. 4 DP states for Text = "xyzcat" (Task 4.2)



Task 4.4 – Regression Check

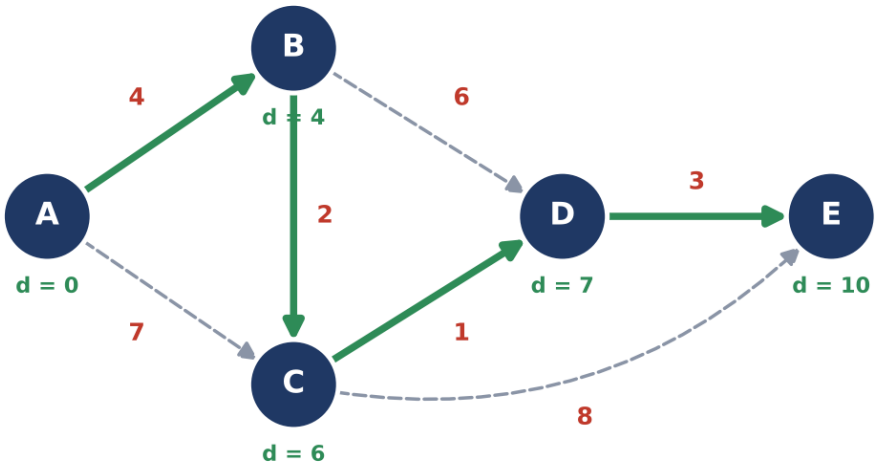
Text = "catdog", Dictionary = { cat, dog } – the original algorithm already returns the correct answer (TRUE).

i	Relevant split	Original DP[i]	Corrected test	Corrected DP[i]
3	j = 0, "cat" ∈ Dict	TRUE	DP[0] = TRUE and "cat" ∈ Dict → true	TRUE
1, 2, 4, 5	no substring ends in a word	FALSE	word test fails → condition false	FALSE
6	j = 3, "dog" ∈ Dict	TRUE	DP[3] = TRUE and "dog" ∈ Dict → true	TRUE

Both versions give DP[6] = TRUE. The extra condition DP[j] = TRUE is satisfied whenever the prefix really is segmentable, so correct behaviour is preserved and only the false positives are removed.

Task 5 – Dijkstra, Bellman-Ford and Floyd

Fig. 5 Given graph with Dijkstra shortest-path tree (Task 5.1)



Green = shortest-path tree from A (A→B→C→D→E, distance 10)

Task 5.1 – Greedy Shortest-Path Design (non-negative weights)

```
ALGORITHM GreedyShortestPath(V, Edges, Weight, Source)
INPUT:  vertex set V, directed edges (u, v) with Weight(u, v) ≥ 0, a Source vertex
OUTPUT: Dist[v] (shortest distance from Source), Pred[v] (predecessor on that path)
BEGIN
  FOR EACH v IN V DO                          // initialisation
    Dist[v] ← ∞;  Pred[v] ← NIL;  Done[v] ← FALSE
  END FOR
  Dist[Source] ← 0
  FOR count ← 1 TO |V| DO
    u ← the vertex with Done[u] = FALSE having the smallest Dist[u]   // selection
    IF Dist[u] = ∞ THEN BREAK                 // remaining vertices unreachable
    Done[u] ← TRUE                            // Dist[u] is now final (greedy commitment)
    FOR EACH edge (u, v) leaving u DO
      IF Done[v] = FALSE AND Dist[u] + Weight(u, v) < Dist[v] THEN   // relaxation
        Dist[v] ← Dist[u] + Weight(u, v)  // distance update
        Pred[v] ← u                       // predecessor update
      END IF
    END FOR
  END FOR                                     // termination: all vertices finalised
  RETURN Dist, Pred                           // or no reachable vertex is left
END
```

Trace from A:

Iter	Selected u	Relaxations	Dist [A, B, C, D, E]	Pred changes
init	–	–	[0, ∞, ∞, ∞, ∞]	–
1	A (0)	B = 0+4 = 4; C = 0+7 = 7	[0, 4, 7, ∞, ∞]	B ← A, C ← A

Iter	Selected u	Relaxations	Dist [A, B, C, D, E]	Pred changes
2	B (4)	C = 4+2 = 6 < 7 ✓; D = 4+6 = 10	[0, 4, 6, 10, ∞]	C ← B, D ← B
3	C (6)	D = 6+1 = 7 < 10 ✓; E = 6+8 = 14	[0, 4, 6, 7, 14]	D ← C, E ← C
4	D (7)	E = 7+3 = 10 < 14 ✓	[0, 4, 6, 7, 10]	E ← D
5	E (10)	no outgoing edges	[0, 4, 6, 7, 10]	–

Result: Dist = A 0, B 4, C 6, D 7, E 10. Paths: A→B (4), A→B→C (6), A→B→C→D (7), A→B→C→D→E (10).

Task 5.2 – Negative-Weight Edges (Bellman-Ford)

Unsafe assumption: the step "Done[u] ← TRUE – Dist[u] is final" when u has the smallest tentative distance. This is valid only if extending a path can never make it shorter, i.e. all weights are ≥ 0. With a negative edge, a vertex that is finalised early may later be reached more cheaply through a vertex not yet processed.

Counter-example: edges A→B = 2, A→C = 5, C→B = −4. Greedy finalises B with 2 first, but the path A→C→B costs 5 − 4 = 1. The commitment to B was wrong.

```
FOR EACH v IN V DO  Dist[v] ← ∞;  Pred[v] ← NIL  END FOR
Dist[Source] ← 0
FOR pass ← 1 TO |V| − 1 DO                    // repeated relaxation
  Changed ← FALSE
  FOR EACH edge (u, v, w) IN Edges DO
    IF Dist[u] ≠ ∞ AND Dist[u] + w < Dist[v] THEN
      Dist[v] ← Dist[u] + w
      Pred[v] ← u
      Changed ← TRUE
    END IF
  END FOR
  IF Changed = FALSE THEN BREAK
END FOR
// ---- negative-cycle detection (one extra pass) ----
FOR EACH edge (u, v, w) IN Edges DO
  IF Dist[u] ≠ ∞ AND Dist[u] + w < Dist[v] THEN
    RETURN "Reachable negative-weight cycle"
  END IF
END FOR
RETURN Dist, Pred
```

Why one greedy commitment is not enough: with negative edges, a distance can still decrease after a vertex has been chosen, so no vertex can be declared final early. Bellman-Ford instead relaxes **all** edges repeatedly. After pass k, every shortest path using at most k edges is correct; since a shortest simple path has at most |V| − 1 edges, |V| − 1 passes suffice.

Detecting a reachable negative cycle: if, after |V| − 1 passes, some edge (u, v, w) can still be relaxed (Dist[u] + w < Dist[v], with Dist[u] finite), then a path with |V| edges is shorter than any simple path. That is only possible if a negative cycle reachable from the source exists.

Task 5.3 – All-Pairs Shortest Paths (Floyd)

State: D^(k)[i][j] = the length of the shortest path from vertex i to vertex j that uses **only vertices 1..k as intermediate vertices** (i and j themselves are free). D^(0)[i][j] is the direct edge weight (0 if i = j, ∞ if there is no edge); D^(n) holds the final answer.

```
D(k)[i][j] = MIN( D(k−1)[i][j] ,  D(k−1)[i][k] + D(k−1)[k][j] )

FOR k ← 1 TO n DO                    // allow vertex k as an intermediate
  FOR i ← 1 TO n DO
    FOR j ← 1 TO n DO
      IF D[i][k] + D[k][j] < D[i][j] THEN
        D[i][j] ← D[i][k] + D[k][j]
      END IF
    END FOR
  END FOR
END FOR
```

Final all-pairs matrix for the given graph (order A, B, C, D, E):

From \ To	A	B	C	D	E
A	0	4	6	7	10
B	∞	0	2	3	6
C	∞	∞	0	1	4
D	∞	∞	∞	0	3
E	∞	∞	∞	∞	0

How the interpretation differs:

Method	What a state means	Nature of the strategy
Dijkstra	Dist[v] = best known distance from the single source; once v is selected it is final	Greedy – commits to the closest unsettled vertex; needs non-negative weights
Bellman-Ford	Dist[v] after pass k = shortest distance from the source using at most k edges	Repeated relaxation – restricts the number of edges; no early commitment
Floyd	D^(k)[i][j] = shortest distance between **every pair** (i, j) using only vertices 1..k as intermediates	Dynamic programming over a growing set of allowed intermediate vertices; handles all pairs and negative edges (no negative cycle)

Task 6 – Design Challenge: Energy-Aware Job Scheduling

Stage 1 – Interpret the Problem

Element	Description
Input	n jobs; job j has Start[j], Finish[j] (Start < Finish) and an energy-saving Value[j] ≥ 0.
Output	A set of selected jobs and their total energy-saving value.
Objective	Maximise the sum of Value[j] over the selected jobs.
Constraints	The server runs one job at a time, so any two selected jobs must not overlap. Jobs touching at an end point (one finishes at t, another starts at t) are compatible.
Decisions	For every job: select it or not. Selecting job j rules out every job that overlaps it.

This is **weighted interval scheduling**: the values are arbitrary weights, not all equal to 1, which matters for the choice of strategy.

Stage 2 – Propose the Strategy

Candidate rule	Why it can fail in general	Counter-example
Earliest finish time first	Optimal only when all values are equal. It ignores value, so a cheap early job can block a valuable one.	A (1–3, value 1) and B (2–6, value 100): picks A, loses B.
Highest value first	A single high-value long job can block several jobs whose combined value is larger.	X (1–10, value 60) versus Y (1–5, value 50) and Z (5–10, value 50): picks 60, optimum is 100.
Shortest duration first	Short jobs can sit in the middle of a long, valuable job and be of low value.	Short job (4–5, value 1) lies inside long job (1–10, value 40): picks 1 and loses 40.
Brute force over all subsets	Correct but needs 2^n subsets, impractical.	–
Dynamic programming (chosen)	Compares the "take" and "skip" outcomes for every job using optimal results of smaller problems; never commits blindly.	No counter-example; correctness follows from the recurrence.

Justification. No simple local rule is safe for arbitrary values, so a greedy choice cannot be defended. The problem has **optimal substructure**: if job j is in an optimal schedule, the rest must be an optimal schedule among the jobs that finish before j starts; if j is not, the rest is optimal among the first j − 1 jobs. It also has **overlapping subproblems** (the same prefix of jobs is needed again and again), so dynamic programming over jobs sorted by finish time solves it in polynomial time. (On the given data, highest-value-first happens to give 200, but that is a coincidence and not a justification.)

Stage 3 – Identify the Critical Information

Sort jobs by **finish time** (ascending) and number them 1..n. Define:

- **p(j)** = the largest index $i < j$ with Finish[i] ≤ Start[j] (0 if none). Because of the sorting, **all** jobs 1..p(j) are compatible with job j, and no job between p(j)+1 and j−1 is.
- **State:** OPT[j] = the maximum total value obtainable from jobs 1..j (a set of non-overlapping jobs).
- **Decision for job j:** skip it, which gives OPT[j−1]; or take it, which gives Value[j] + OPT[p(j)].
- **Recurrence:** OPT[j] = max(OPT[j−1] , Value[j] + OPT[p(j)]).
- **Smaller results required:** OPT[j−1] and OPT[p(j)], both with indices smaller than j.
- **Base condition:** OPT[0] = 0 (no jobs, no value).
- **Feasibility information:** Start[j] and the finish times of earlier jobs, captured in p(j).

Stage 4 – Complete Pseudocode

```
ALGORITHM EnergyAwareScheduling(n, JobID[1..n], Start[1..n], Finish[1..n], Value[1..n])
INPUT:  n jobs, Start[j] < Finish[j], Value[j] ≥ 0
OUTPUT: Selected (job IDs in time order), MaxValue
BEGIN
  // ---- preprocessing ----
  SORT the jobs in ascending order of Finish   (carry JobID, Start, Value along)
  FOR j ← 1 TO n DO                          // compatible predecessor p(j)
    P[j] ← 0
    FOR i ← j − 1 DOWN TO 1 DO
      IF Finish[i] ≤ Start[j] THEN            // touching jobs are compatible
        P[j] ← i
        BREAK                                 // largest such i found
      END IF
    END FOR
  END FOR
  // ---- initialisation ----
  OPT[0] ← 0
  // ---- decision logic and state updates ----
  FOR j ← 1 TO n DO
    Skip ← OPT[j − 1]
    Take ← Value[j] + OPT[P[j]]               // jobs overlapping j are excluded by P[j]
    IF Take > Skip THEN
      OPT[j] ← Take;  Choice[j] ← TAKE
    ELSE
      OPT[j] ← Skip;  Choice[j] ← SKIP        // ties: prefer skipping
    END IF
  END FOR
  // ---- reconstruction ----
  Selected ← EMPTY LIST
  j ← n
  WHILE j > 0 DO                              // termination when j reaches 0
    IF Choice[j] = TAKE THEN
      INSERT JobID[j] at the FRONT of Selected
      j ← P[j]                                // jump over incompatible jobs
    ELSE
      j ← j − 1
    END IF
  END WHILE
  MaxValue ← OPT[n]
  RETURN Selected, MaxValue
END
```

Complexity: sorting O(n log n); computing p(j) $O(n^2)$ as written (O(n log n) with binary search); DP loop O(n); reconstruction O(n).

Stage 5 – Desk-Check on the Eight-Job Dataset

Jobs are already in ascending order of finish time (3, 5, 6, 8, 9, 10, 11, 13), so job numbers 1..8 equal J1..J8.

j	Job	Start–Finish	Value	p(j)	Skip = OPT[j−1]	Take	OPT[j]	Choice
1	J1	1–3	20	0	0	20 + OPT[0] = 20	20	TAKE
2	J2	2–5	50	0	20	50 + OPT[0] = 50	50	TAKE
3	J3	4–6	30	1	50	30 + OPT[1] = 50	50	SKIP (tie)
4	J4	6–8	40	3	50	40 + OPT[3] = 90	90	TAKE
5	J5	5–9	60	2	90	60 + OPT[2] = 110	110	TAKE
6	J6	8–10	30	4	110	30 + OPT[4] = 120	120	TAKE
7	J7	9–11	50	5	120	50 + OPT[5] = 160	160	TAKE
8	J8	11–13	40	7	160	40 + OPT[7] = 200	200	TAKE

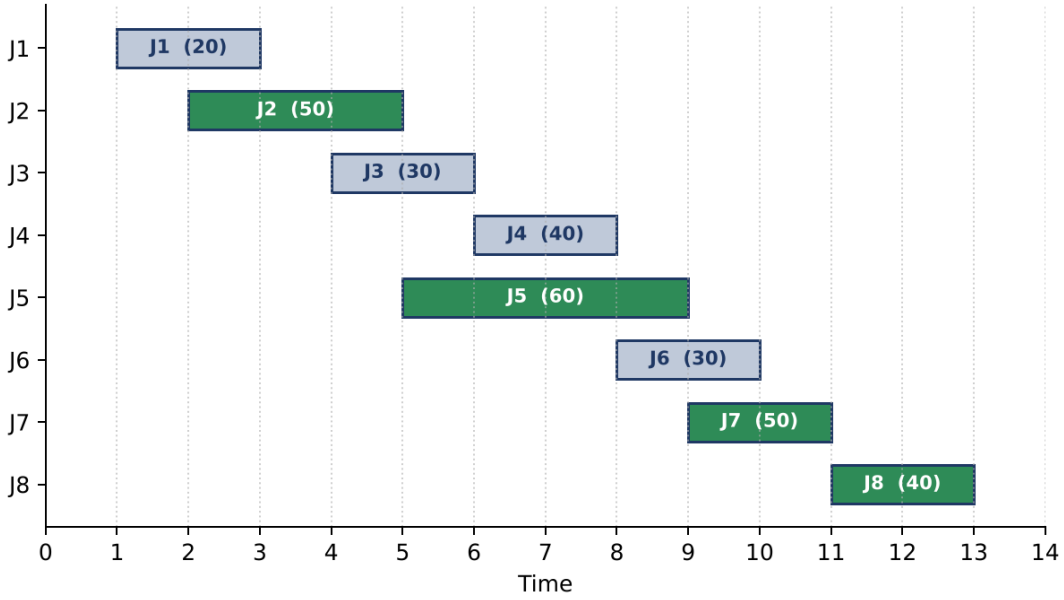
Example for p(j): for J6 (start 8) the largest earlier job finishing by time 8 is J4 (finish 8), so p = 4. J7 (start 9) has p = 5 because J5 finishes at 9 (touching is allowed).

Reconstruction (from j = 8):

j	Choice[j]	Action	Next j
8	TAKE (200 > 160)	Select J8	p(8) = 7
7	TAKE (160 > 120)	Select J7	p(7) = 5
5	TAKE (110 > 90)	Select J5	p(5) = 2
2	TAKE (50 > 20)	Select J2	p(2) = 0 → stop

Jobs selected: J2 (2–5), J5 (5–9), J7 (9–11), J8 (11–13). Total energy-saving value = 50 + 60 + 50 + 40 = 200. The chosen jobs meet only at end points (5, 9, 11), so there is no overlap.

Fig. 6 Eight-job dataset – selected jobs J2, J5, J7, J8 (green), total 200



Stage 6 – Attack Your Own Design

Adversarial job set (5 jobs):

Job	Start	Finish	Value
T1	1	3	5
T2	2	6	100
T3	1	10	110
T4	6	10	30
T5	10	12	20

Why this input is challenging: (1) T3 has the highest value (110) but is long and blocks T1, T2 and T4; (2) T1 finishes earliest but is worth only 5 and conflicts with the valuable T2; (3) T2 ends exactly when T4 starts, and T3/T4 end exactly when T5 starts, testing the "touching jobs are compatible" rule; (4) T3 and T4 have the **same finish time**, testing sorting ties and p(j).

Prediction: earliest-finish greedy will choose T1 then T4, T5 (value 55); highest-value greedy will choose T3 then T5 (130). My DP should skip the attractive T3 and instead take T2, T4, T5 for 150. The critical decision should be at T3, where Skip must beat Take.

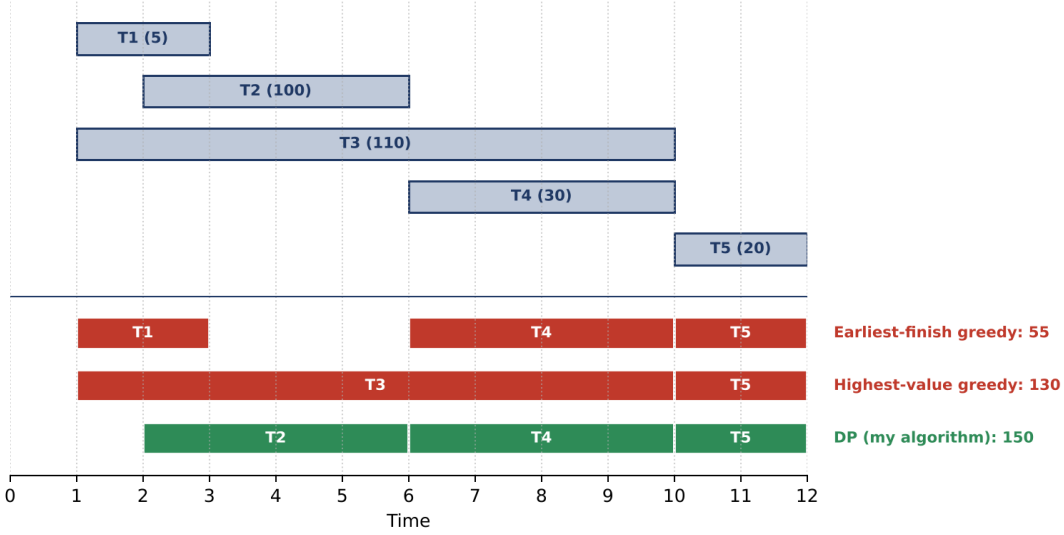
Application of the algorithm. Sorted by finish time: 1 = T1 (1–3), 2 = T2 (2–6), 3 = T4 (6–10), 4 = T3 (1–10), 5 = T5 (10–12).

j	Job	Value	p(j)	Skip	Take	OPT[j]	Choice
1	T1	5	0	0	5 + 0 = 5	5	TAKE
2	T2	100	0	5	100 + 0 = 100	100	TAKE
3	T4	30	2	100	30 + OPT[2] = 130	130	TAKE
4	T3	110	0	130	110 + 0 = 110	130	**SKIP**
5	T5	20	4	130	20 + OPT[4] = 150	150	TAKE

Reconstruction: j = 5 TAKE → T5, j = p(5) = 4; j = 4 SKIP → j = 3; j = 3 TAKE → T4, j = p(3) = 2; j = 2 TAKE → T2, j = 0. Selected jobs **T2 (2–6), T4 (6–10), T5 (10–12)**, value 100 + 30 + 20 = **150**.

Strategy	Jobs chosen	Total value
Earliest finish first	T1, T4, T5	55
Highest value first	T3, T5	130
My DP algorithm	T2, T4, T5	**150 (optimal)**

Fig. 7 Adversarial test set and the schedules chosen by each strategy (Stage 6)



Conclusion. The algorithm remained optimal; the prediction was correct. The critical decision was j = 4 (T3): the locally attractive highest-value job (110) lost to the combination already available, OPT[3] = 130. No assumption fails because:

- The DP never commits to a job on local attractiveness; it compares **Take** and **Skip** for every job.
- Take uses OPT[p(j)], and every job from 1..p(j) is compatible with j because the list is sorted by finish time; jobs after p(j) that overlap j are correctly excluded.
- For equal finish times (T4, T3), the later index is used when p(T5) is computed; OPT is non-decreasing, so this loses nothing, and touching jobs (finish = start) are accepted by the test Finish[i] ≤ Start[j].
- Optimal substructure gives correctness by induction on j: an optimal schedule for jobs 1..j either contains j (and then its remainder is optimal for 1..p(j)) or does not (and is optimal for 1..j−1).